

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA1412 COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 20%

Duration :60 Mins

On Class QUESTIONS: 5 Total Marks:50

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

Code of Conduct:

I Bhavya S(192524376) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Bhavya S

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

Q1. Smart Retail Billing System

The given billing equations are:

$\text{total_cost} = \text{price} * \text{quantity}$

$\text{discount_value} = \text{total_cost} * \text{discount}$

$\text{final_bill} = \text{total_cost} - \text{discount_value}$

(i) Declarations of variables

Suitable data types:

float price;

int quantity;

float discount;

float total_cost;

float discount_value;

float final_bill;

- price → float, because product price may contain decimals.
- quantity → int, because quantity is normally a whole number.
- discount → float, because discount may be fractional/percentage.
- total_cost, discount_value, final_bill → float, because monetary values may contain decimals

(ii) Three-Address Code (TAC)

Three-address code uses temporary variables so that each instruction contains at most three operands.

Step Three-Address Code

1 t1 = price * quantity

2 total_cost = t1

3 t2 = total_cost * discount

4 discount_value = t2

5 t3 = total_cost - discount_value

6 final_bill = t3

Therefore, the TAC is:

t1 = price * quantity

total_cost = t1

t2 = total_cost * discount

discount_value = t2

t3 = total_cost - discount_value

final_bill = t3

(iii) Quadruples, Triples and Indirect Triples

A. Quadruples

A quadruple contains:

(Operator, Argument 1, Argument 2, Result)

No. Operator Arg1 Arg2 Result

1 * price quantity t1
2 = t1 — total_cost
3 * total_cost discount t2
4 = t2 — discount_value
5 - total_cost discount_value t3
6 = t3 — final_bill

B. Triples

Triples do not explicitly use temporary variable names. Results are referred to by their instruction numbers.

No. Operator Arg1 Arg2

0 * price quantity
1 = (0) total_cost
2 * total_cost discount
3 = (2) discount_value
4 - total_cost discount_value
5 = (4) final_bill

Here, (0) means the result produced by instruction 0.

C. Indirect Triples

Indirect triples use a separate pointer table to refer to triple statements. **Triple table:**

Triple No. Operator Arg1 Arg2

0 * price quantity
1 = (0) total_cost
2 * total_cost discount
3 = (2) discount_value
4 - total_cost discount_value
5 = (4) final_bill

Pointer table:

Pointer Points to

P0 0

P1 1

P2 2

P3 3

P4 4

P5 5

The pointer table allows the order of execution to be changed without modifying the actual triple statements.

(iv) Optimization opportunities

The generated code contains unnecessary temporary variables and assignment statements. Original:

```
t1 = price * quantity
```

```
total_cost = t1
```

```
t2 = total_cost * discount
```

```
discount_value = t2
```

```
t3 = total_cost - discount_value
```

```
final_bill = t3
```

Possible optimization:

1. **Temporary variable reduction** – unnecessary temporaries can be removed.
2. **Copy propagation** – assignments such as `total_cost = t1` can be incorporated into subsequent calculations.
3. **Avoid unnecessary assignments** – intermediate copies need not always be generated.
4. **Fewer instructions** – reducing temporary variables decreases the number of intermediate-code instructions.

(v) Optimized Intermediate Code and Justification

An optimized form is:

```
total_cost = price * quantity
```

```
discount_value = total_cost * discount
```

```
final_bill = total_cost - discount_value
```

Comparison

Original TAC Optimized TAC

t1 = price * quantity total_cost = price * quantity

total_cost = t1 —

t2 = total_cost * discount discount_value = total_cost * discount

discount_value = t2 —

t3 = total_cost - discount_value final_bill = total_cost - discount_value

final_bill = t3 —

6 instructions 3 instructions

Justification

The optimized code produces the same final bill while eliminating unnecessary temporary variables and copy assignments. Therefore, it reduces the number of intermediate-code instructions, simplifies code generation, and can improve execution efficiency. **Final result:** The optimized intermediate code is:

total_cost = price * quantity

discount_value = total_cost * discount

final_bill = total_cost - discount_value

This addresses all five required parts of Q1 and provides the method, systematic representation, optimization, and justification expected for the highest rubric level.

Q2. University Scholarship Eligibility System

The scholarship eligibility condition is: **CGPA > 8.5 AND (Family Income < 300000 OR Sports Quota = TRUE)**. The question specifically requires Boolean formulation, short circuit intermediate code, backpatching, labelled code, and a control-flow graph. **(i)**

Boolean Expression

Let:

- C = CGPA > 8.5
- I = Family_Income < 300000
- S = Sports_Quota == TRUE

Therefore, the Boolean expression is:

C AND (I OR S)

or

(CGPA > 8.5) AND

(Family_Income < 300000 OR Sports_Quota == TRUE)

A student is eligible **only when CGPA is greater than 8.5**, and at least one of the following is true:

- Family income is less than ₹3,00,000, or
- Student belongs to the sports quota.

(ii) Intermediate Code Using Short-Circuit Evaluation

In short-circuit evaluation:

- For AND, if the first condition is false, the remaining condition need not be evaluated. •

For OR, if the first condition is true, the second condition need not be evaluated. **Unlabelled**

Three-Address Code

1. if CGPA > 8.5 goto 3
2. goto 8
3. if Family_Income < 300000 goto 7
4. if Sports_Quota == TRUE goto 7
5. goto 8
6. goto 8
7. Eligible = TRUE
8. Eligible = FALSE

A cleaner representation using symbolic labels is:

if CGPA > 8.5 goto L1

goto Lfalse

L1:

if Family_Income < 300000 goto Ltrue

if Sports_Quota == TRUE goto Ltrue

goto Lfalse

Ltrue:

Eligible = TRUE

goto Lend

Lfalse:

Eligible = FALSE

Lend:

(iii) Backpatching

Backpatching is used to fill the targets of jumps that are not known when the intermediate code is initially generated.

Initially:

1. if CGPA > 8.5 goto _
2. goto _
3. if Family_Income < 300000 goto _
4. if Sports_Quota == TRUE goto _
5. goto _

Backpatching Lists

For the complete expression:

(CGPA > 8.5) AND

(Family_Income < 300000 OR Sports_Quota == TRUE)

- CGPA > 8.5 true → proceed to income condition.
- CGPA > 8.5 false → FALSE.
- Family_Income < 300000 true → TRUE.
- If income condition is false → check sports quota.
- Sports quota true → TRUE.
- Sports quota false → FALSE.

After backpatching:

1. if CGPA > 8.5 goto 3
2. goto 7
3. if Family_Income < 300000 goto 6
4. if Sports_Quota == TRUE goto 6
5. goto 7
6. Eligible = TRUE
7. Eligible = FALSE

(iv) Final Labelled Intermediate Code

L1:

if CGPA > 8.5 goto L2
goto Lfalse

L2:

if Family_Income < 300000 goto Ltrue
if Sports_Quota == TRUE goto Ltrue

goto Lfalse

Ltrue:

Eligible = TRUE

goto Lend

Lfalse:

Eligible = FALSE

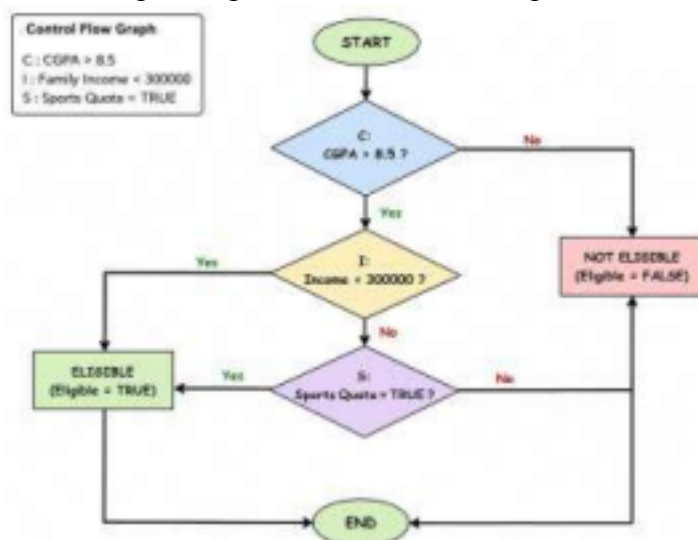
Lend:

Explanation

- **L1:** Start checking CGPA.
- **L2:** CGPA is valid, so check family income. •
- If income is below ₹3,00,000 → eligible. •
- Otherwise, check sports quota.
- If sports quota is true → eligible.
- If both income and sports quota fail → not eligible. •
- Lend:** End of eligibility checking.

(v) Control-Flow Graph

The corresponding control flow can be represented as:



Final Interpretation

The short-circuit implementation avoids unnecessary condition checking. If **CGPA ≤ 8.5**, the income and sports-quota conditions are not required because the complete AND expression is already false. If the income is below ₹3,00,000, the sports-quota condition is skipped because

the OR expression is already true. This reduces unnecessary intermediate-code execution and correctly represents the scholarship eligibility logic.

Q3. Smart Parking Management System

The system assigns parking zones according to vehicle_type: **1 → TWO_WHEELER, 2 → CAR, 3 → BUS, 4 → EMERGENCY_VEHICLE**. The question requires conditional-jump TAC, jump-table implementation, quadruples, comparison, and recommendation. **(i)**

Three-Address Code Using Conditional Jumps

The given switch statement can be converted into a sequence of conditional jumps.

```
if vehicle_type == 1 goto L1
if vehicle_type == 2 goto L2
if vehicle_type == 3 goto L3
if vehicle_type == 4 goto L4
goto L5
L1:
zone = TWO_WHEELER
goto L5
L2:
zone = CAR
goto L5
L3:
zone = BUS
goto L5
L4:
zone = EMERGENCY_VEHICLE
goto L5
L5:
```

Explanation

- The value of vehicle_type is compared with each case value.
- When a match is found, control jumps to the corresponding label.
- After assigning the parking zone, control goes to L5, the end of the switch.
- If none of the cases matches, control directly goes to L5.

(ii) Equivalent Jump-Table Implementation

A jump table stores the target address corresponding to each case value.

if vehicle_type < 1 goto Ldefault

if vehicle_type > 4 goto Ldefault

goto JumpTable[vehicle_type]

JumpTable:

1 → L1

2 → L2

3 → L3

4 → L4

L1:

zone = TWO_WHEELER

goto Lend

L2:

zone = CAR

goto Lend

L3:

zone = BUS

goto Lend

L4:

zone = EMERGENCY_VEHICLE

goto Lend

Ldefault:

zone = INVALID

Lend:

Working

If vehicle_type = 3, the jump table directly selects:

JumpTable[3] → L3

and executes:

zone = BUS

Thus, the jump table avoids checking cases one by

one. (iii) Quadruple Representation

A quadruple has the form:

(Operator, Argument 1, Argument 2, Result)

Conditional-Jump Quadruples

No. Operator Arg1 Arg2 Result 1 IF== vehicle_type

1 L1 2 IF== vehicle_type 2 L2 3 IF== vehicle_type 3

L3 4 IF== vehicle_type 4 L4 5 GOTO — — L5 6 =

TWO_WHEELER — zone 7 GOTO — — L5 8 =

CAR — zone 9 GOTO — — L5 10 = BUS — zone 11

GOTO — — L5 12 = EMERGENCY_VEHICLE —

zone 13 GOTO — — L5

(iv) Conditional Branching vs Jump Table

Feature Conditional Branching Jump Table Working Checks cases one by one

Directly jumps to target

Number of

comparisons May require many comparisons Very few

Execution time Can increase with number of

cases Generally faster for many cases

Memory Requires less additional memory Requires jump-table memory Best suited for

Small number of cases Large number of consecutive cases Implementation Simple Slightly more complex

Parking example Checks 1, then 2, then 3, then 4 Directly accesses corresponding case

Example

For:

vehicle_type = 4

Conditional branching may perform:

4 == 1 ? No

4 == 2 ? No

4 == 3 ? No

4 == 4 ? Yes

The jump table can directly perform:

JumpTable[4] → L4

Therefore, the jump-table approach can reduce unnecessary

comparisons. **(v) Best Approach for a Large-Scale Parking System**

For a large-scale parking management system, the jump-table implementation is the better approach when vehicle types/case values are numerous and reasonably consecutive.

Justification

1. **Faster selection:** It can directly locate the required case.
2. **Fewer comparisons:** It avoids checking every case sequentially.
3. **Scalability:** It is more suitable when the number of vehicle categories increases.
4. **Efficient execution:** Direct indexing can reduce branching overhead.
5. **Suitable for frequent processing:** Parking systems may process a large number of vehicles, so faster case selection is beneficial.

Conclusion

For a small number of vehicle types, conditional branching is simple and easy to implement. For a large-scale parking system with many case values, a jump table is preferable because it provides faster and more direct case selection.

Q4. Online Food Delivery Processing System

The system executes these procedure calls:

amount = calculateBill(order)

applyCoupon(amount)

generateInvoice(amount)

The question covers procedure calls, parameter passing, return values, activation records, and stack operations.

(i) Intermediate Code for Procedure Calls

The corresponding three-address intermediate code is:

param order

t1 = call calculateBill, 1

amount = t1

param amount

call applyCoupon, 1

param amount

call generateInvoice, 1

Explanation

- param order passes the order information to calculateBill().
- call calculateBill, 1 invokes the procedure with one parameter.
- The returned bill amount is stored in temporary variable t1.
- amount = t1 stores the returned value.
- amount is then passed to applyCoupon().
- Finally, the amount is passed to generateInvoice().

(ii) Parameter Passing and Return Value Handling

calculateBill(order)

param order

t1 = call calculateBill, 1

amount = t1

Here:

- order is the **actual parameter**.
- The procedure receives order as its parameter.
- calculateBill() returns a value.
- The returned value is stored in t1.
- amount receives the returned value.

applyCoupon(amount)

param amount

call applyCoupon, 1

The current amount is passed to applyCoupon().

generateInvoice(amount)

param amount

call generateInvoice, 1

The final amount is passed to generateInvoice() to generate the invoice. **Parameter Flow**

order

↓

calculateBill(order)

↓

returned amount

↓

applyCoupon(amount)

↓

generateInvoice(amount)

(iii) Activation Record / Stack Frame

An **activation record** is the stack frame created when a procedure is called. It stores information required for execution and return.

Activation Record for calculateBill()

Field Purpose

Parameters order

Return Value Calculated bill amount

Return Address Address to return to caller

Field Purpose

Local Variables Variables used inside calculateBill() Control

Information Information required to restore previous execution

Activation Record for applyCoupon()

Field Purpose

Parameters amount

Return Value If any, updated/processed amount

Return Address Address of caller

Local Variables Coupon-related temporary variables

Control Information Previous execution information

Activation Record for generateInvoice()

Field Purpose

Parameters amount

Return Value If any

Return Address Address of caller

Local Variables Invoice-related variables

Control Information Previous execution information

(iv) Sequence of Call and Return Operations Using

Stack The execution takes place through stack frames.

Step 1: Call `calculateBill(order)`

STACK

calculateBill()
Parameter: order
Return Address
Local Variables

After execution, `calculateBill()` returns the calculated amount.

amount = returned value

Step 2: Call `applyCoupon(amount)`

STACK

applyCoupon()
Parameter: amount
Return Address
Local Variables

After the procedure finishes, its activation record is removed from the stack.

Step 3: Call `generateInvoice(amount)`

STACK

generateInvoice()
Parameter: amount
Return Address
Local Variables

After invoice generation, the procedure returns and its activation record is removed. **Overall Stack Sequence**

main

↓
 calculateBill(order)
 ↓
 return amount
 ↓
 applyCoupon(amount)
 ↓
 return
 ↓
 generateInvoice(amount)
 ↓
 return
 ↓
 main

(v) Storage of Local Variables, Parameters and Return Addresses

The activation record provides separate storage for the information required by each procedure.

Component Stored Information

Parameters Input values passed to the procedure, such as order and amount **Local**

Variables Temporary variables created and used within the procedure

Return Address Address of the instruction where execution should continue after the procedure returns

Control

Information Information required to restore the caller's execution environment

Return Value Value produced by a procedure, such as the bill calculated by calculateBill()

Final Explanation

When calculateBill(order) is called, an activation record is pushed onto the stack containing its parameter, local information, and return address. After calculating the bill, the return value is passed back and the activation record is removed. The same process occurs for applyCoupon() and generateInvoice(). Thus, activation records provide an organized way to

manage parameters, local variables, return addresses, and procedure execution on the runtime stack.

Q5. Smart Industrial Safety Monitoring System

The safety system activates the emergency alarm when **Temperature > 100 AND (Pressure > 80 OR GasLeak = TRUE)**. If the condition is satisfied, it calls `activateAlarm()` and `sendNotification()`.

(i) Boolean Expression

Let:

- $T = \text{Temperature} > 100$
- $P = \text{Pressure} > 80$
- $G = \text{GasLeak} == \text{TRUE}$

Therefore, the Boolean expression is:

$(T > 100) \text{ AND } ((P > 80) \text{ OR } (G == \text{TRUE}))$

More precisely using the defined conditions:

$(\text{Temperature} > 100) \text{ AND } (\text{Pressure} > 80 \text{ OR } \text{GasLeak} == \text{TRUE})$

The alarm is activated only when the temperature condition is true and at least one of the pressure or gas-leak conditions is true.

(ii) Intermediate Code Using Short-Circuit Evaluation

For:

$\text{Temperature} > 100 \text{ AND } (\text{Pressure} > 80 \text{ OR } \text{GasLeak} == \text{TRUE})$

short-circuit evaluation works as follows:

- If $\text{Temperature} \leq 100$, the complete condition is false.
- If $\text{Temperature} > 100$, check pressure.
- If $\text{Pressure} > 80$, the OR condition is already true, so gas leak need not be checked. •
If pressure is not greater than 80, check GasLeak.
- If both pressure and gas leak are false, the condition is false.

Three-Address Code

if $\text{Temperature} > 100$ goto L1

goto Lfalse

L1:

if $\text{Pressure} > 80$ goto Ltrue

if GasLeak == TRUE goto Ltrue

goto Lfalse

Ltrue:

call activateAlarm

call sendNotification

goto Lend

Lfalse:

goto Lend

Lend:

(iii) Backpatching

Initially, the jump targets can be left incomplete:

1. if Temperature > 100 goto _

2. goto _

3. if Pressure > 80 goto _

4. if GasLeak == TRUE goto _

5. goto _

6. call activateAlarm

7. call sendNotification

8. goto _

After applying backpatching, the forward jumps are resolved:

Instruction Intermediate Code Target

1 if Temperature > 100 goto L1 L1

2 goto Lfalse Lfalse

3 if Pressure > 80 goto Ltrue Ltrue

4 if GasLeak == TRUE goto Ltrue Ltrue

5 goto Lfalse Lfalse

Thus, the backpatched code becomes:

if Temperature > 100 goto L1

goto Lfalse

L1:

if Pressure > 80 goto Ltrue

if GasLeak == TRUE goto Ltrue

goto Lfalse

(iv) Intermediate Code for Procedure Calls

When the safety condition is true, the following procedures are executed: Ltrue:

call activateAlarm

call sendNotification

goto Lend

The complete intermediate code is:

if Temperature > 100 goto L1

goto Lfalse

L1:

if Pressure > 80 goto Ltrue

if GasLeak == TRUE goto Ltrue

goto Lfalse

Ltrue:

call activateAlarm

call sendNotification

goto Lend

Lfalse:

goto Lend

Lend:

Procedure Execution

1. activateAlarm() activates the emergency alarm.
2. sendNotification() sends an alert/notification.
3. Control then proceeds to Lend.

(v) Complete Labelled Control Flow and Execution

Sequence Labelled Intermediate Code

L0:

if Temperature > 100 goto L1

goto Lfalse

L1:

if Pressure > 80 goto Ltrue

if GasLeak == TRUE goto Ltrue

```

goto Lfalse
Ltrue:
call activateAlarm
call sendNotification
goto Lend
Lfalse:
goto Lend
Lend:

```

Execution Sequence

Case 1: Temperature ≤ 100

L0 $\rightarrow$ Lfalse $\rightarrow$ Lend

No alarm is activated.

Case 2: Temperature > 100 and Pressure > 80

L0 $\rightarrow$ L1 $\rightarrow$ Ltrue

$\rightarrow$ activateAlarm()

$\rightarrow$ sendNotification()

$\rightarrow$ Lend

The gas-leak condition is skipped because the OR condition is already true.

Case 3: Temperature > 100 , Pressure ≤ 80 , GasLeak = TRUE

L0 $\rightarrow$ L1 $\rightarrow$ check GasLeak

$\rightarrow$ Ltrue

$\rightarrow$ activateAlarm()

$\rightarrow$ sendNotification()

$\rightarrow$ Lend

Case 4: Temperature > 100 , Pressure ≤ 80 , GasLeak = FALSE

L0 $\rightarrow$ L1 $\rightarrow$ Lfalse $\rightarrow$ Lend

No alarm is activated.

Control Flow

START

|

v

Temperature > 100 ?

/\

```

Yes No
||
v v
Pressure > 80? Lfalse
/\
Yes No
||
| v
| GasLeak = TRUE?
| /\
| Yes No
|||
v v v
Ltrue Lfalse
|
v
activateAlarm()
|
v
sendNotification()
|
v
END

```

Final Interpretation

The intermediate code uses short-circuit evaluation to avoid unnecessary condition checks. If the temperature is not above 100, the remaining conditions are skipped. If pressure is above 80 after the temperature condition is satisfied, the gas-leak condition is also skipped because the OR condition is already true. When the complete safety condition is satisfied, both `activateAlarm()` and `sendNotification()` are executed. This provides an efficient representation of the industrial safety logic.